PRATHYUSHA ENGINEERING COLLEGE

(An Autonomous Institution)

TIRUVALLUR – 602025

COMPUTER ORGANIZATION IN DRONES

CASE STUDY

OF

241GES301J – DIGITAL PRINCIPLES AND COMPUTER ORGANIZATION

THE BACHELOR OF ENGINEERING (B.E)

IN

CSE-(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

SUBMITTED BY

RITHIKA.K - 111425148061
ROSHINI.V - 111425148062
RUPESH.R - 111425148063
SARAN KARTHIKEYAN.V - 111425148064
SAVITHA.B - 111425148065
SHAIK ASGAR - 111425148066

INTRODUCTION

An unmanned aerial vehicle (UAV), commonly called a drone, is an aircraft that flies without a human pilot on board. It is controlled either remotely by a person on the ground or autonomously by onboard computers that follow a programmed mission. Drones are used today in aerial photography, agriculture, surveying and mapping, parcel delivery, infrastructure inspection, search and rescue, firefighting, defence, and academic research. Behind all these applications is a small but very capable embedded computer system that senses the environment, makes decisions, and drives the motors hundreds of times every second.

Computer organization deals with the operational units of a computer and their interconnections: how the processor, memory, input/output devices and buses are arranged and how they cooperate to execute instructions. It is closely related to **digital principles**, because every unit inside a computer is built from logic gates, flip-flops, registers, counters, multiplexers and arithmetic circuits. A drone is an excellent case study for both subjects because it is a complete real-time digital system in which these building blocks are clearly visible.

A typical multirotor drone contains a **flight controller**, which is a microcontroller-based computer board. The flight controller reads data from sensors such as the gyroscope, accelerometer, magnetometer, barometer and GPS receiver. It processes this data using a fixed-point or floating-point processor, compares the current state of the aircraft with the desired state, and calculates the speed required for each motor. The result is sent as digital pulses to the Electronic Speed Controllers (ESCs), which drive the brushless motors. Unlike a general-purpose computer or a smartphone, a drone computer must meet strict **real-time** deadlines. If the control calculation is delayed even by a few milliseconds, the aircraft can become unstable and crash. The system must also be light, consume little power, tolerate vibration and temperature variation, and continue to operate safely when a sensor or communication link fails.

Advanced drones also carry a second computer called a **companion computer**. It runs heavy tasks such as computer vision, obstacle avoidance, object tracking, mapping and machine-learning inference, while the flight controller remains responsible for stability. This two-level organization is a practical example of dividing work among specialized processors, an idea that also appears in the System-on-Chip designs of modern smartphones.

This case study examines the computer organization of drones: the processor, memory, input/output, buses, timers, sensors, software and power system, and shows how the principles of digital logic and computer organization are applied in a real flying machine.

BACKGROUND

The history of unmanned aircraft goes back to the early twentieth century, when simple radio-controlled aircraft were built mainly as military target vehicles. Their control systems were analog: relays, vacuum tubes and, later, discrete transistor circuits. The pilot's commands were converted directly into movement of the control surfaces, and there was no onboard computation of any significant kind.

The arrival of the microprocessor and later the microcontroller changed this situation. Small digital processors could read sensors, execute control algorithms and generate output signals on a single chip. The development of **MEMS** (Micro-Electro-Mechanical Systems) sensors, which place a gyroscope and accelerometer on a tiny silicon chip, allowed an inertial measurement unit to weigh only a fraction of a gram. Together with high-energy-density lithium-polymer batteries and efficient brushless motors, these technologies made small multirotor drones practical and affordable. Since a quadcopter is naturally unstable, it can fly only because a computer continuously corrects its attitude. Modern flight controllers are built around 32-bit ARM Cortex-M microcontrollers, commonly from the Cortex-M4, M7 and similar families, running at clock frequencies of the order of a few hundred megahertz. Open-source autopilot projects such as ArduPilot and PX4 run on these boards, and many commercial drones use similar designs. For vision and AI tasks, drones increasingly carry a companion computer based on a multi-core application processor with a GPU, such as a single-board computer or an embedded AI module.

Main Hardware Units of a Drone

The principal units of a drone's computer system and their roles in computer organization are summarised in the table below.

Unit	Role in Computer Organization	Typical Implementation
Flight controller (CPU)	Executes control program, performs arithmetic and logic operations	32-bit ARM Cortex-M microcontroller with FPU
Memory	Stores program (non-volatile) and working data (volatile)	On-chip Flash and SRAM; external SD card for logs
Input devices	Provide measurements of the environment	IMU, magnetometer, barometer, GNSS, camera, RC receiver
Output devices	Convert digital results into physical action	ESCs, brushless motors, LEDs, buzzer, gimbal servos
Buses / interfaces	Carry data between processor, memory and peripherals	Internal system bus; SPI, I2C, UART, CAN

Unit	Role in Computer Organization	Typical Implementation
Timers	Generate precise time bases and control pulses	Hardware timers producing PWM / DShot signals
Companion computer	Handles heavy computing such as vision and AI	Single-board computer or AI module with GPU/NPU
Power system	Supplies regulated voltage to all digital circuits	LiPo battery, power distribution board, voltage regulator

The flight controller is the heart of this system. It follows the stored-program concept of the von Neumann computer: instructions are fetched from memory, decoded by the control unit, and executed by the arithmetic and logic unit (ALU), and the results are stored in registers or memory. Most microcontroller cores use a modified Harvard organization, with separate paths for instructions and data inside the chip, so that the next instruction can be fetched while data from the previous one is being accessed.

A second important background idea is that of the **embedded system**. A drone computer has a fixed purpose and is permanently built into the machine it controls. It has no keyboard or monitor and typically no conventional operating system. Its software is designed around the hardware, and its performance is judged by determinism and reliability rather than by raw speed.

Comparison with a Smartphone

A drone's flight controller and a smartphone SoC both integrate processor cores, memory and many peripherals, but they are optimized for different goals, as shown below.

Feature	Smartphone SoC	Drone Flight Controller
Primary goal	Rich applications, multimedia, user interface	Stable real-time control of flight
Processor	Multi-core, GHz-class, application processor	Single 32-bit microcontroller, hundreds of MHz
Operating system	Android / iOS (general purpose)	Real-time operating system or bare-metal firmware
Timing	Best effort; small delays are tolerable	Hard deadlines; delays can cause a crash
Memory	Gigabytes of RAM and storage	Kilobytes to a few megabytes on-chip

PROBLEM STATEMENT

A drone must stay balanced against wind, follow commands, and often perform tasks such as mapping or obstacle avoidance, while carrying its own battery. Its computer system must therefore be light, compact and economical in power, which is a difficult design problem. The main problem can be stated as: *How can the computer organization of a drone provide fast and deterministic real-time control, reliable sensor and communication interfaces, and enough processing capability for advanced tasks such as navigation and vision, while keeping weight, power consumption, cost and heat to a minimum, and while remaining safe in the presence of faults?*

Several specific problems arise from this requirement:

- **Real-time deadlines:** The control loop must run at a fixed rate with small, predictable delay; unpredictable delay reduces stability.
- **Limited power and weight:** A larger battery adds weight, which needs more power, so the processor must give the best performance per watt.
- **Limited memory:** A microcontroller has little SRAM and Flash, so software must use memory economically.
- **Noisy sensor data:** MEMS sensors have noise and drift; the digital system must filter and combine the data.
- **Safety and fault tolerance:** Software hangs, sensor failures, signal loss and low battery must be handled safely.

Drone computer organization must therefore balance several competing requirements, as shown in the figure below.

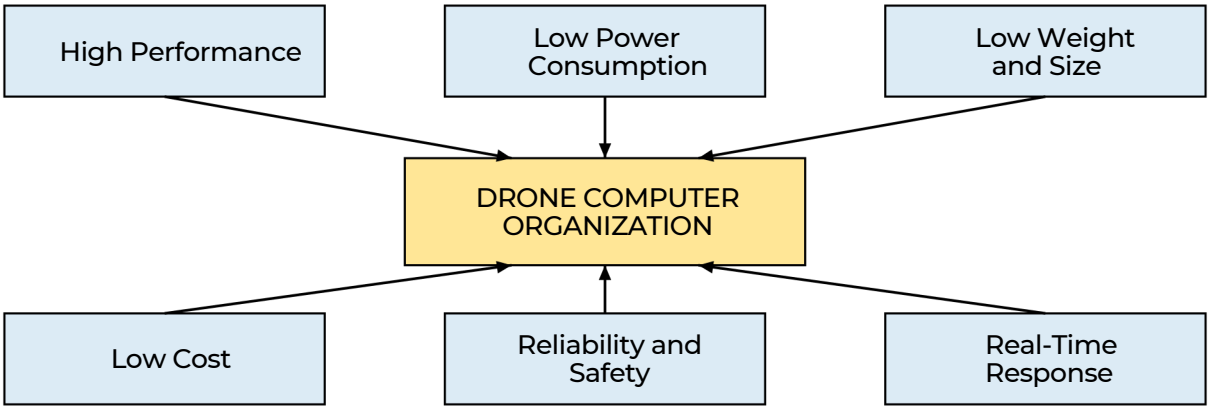


Figure 1: Competing design requirements

ANALYSIS

1. Overall System Organization

The organization of a typical quadcopter is shown in Figure 2. The flight controller is at the centre. Sensors feed data into it, the radio link brings commands from the pilot or ground station, and its outputs go to the four ESCs that drive the motors. A battery and power distribution board supply energy to all units, and a storage device records flight data. A companion computer, when present, communicates with the flight controller over a serial link.

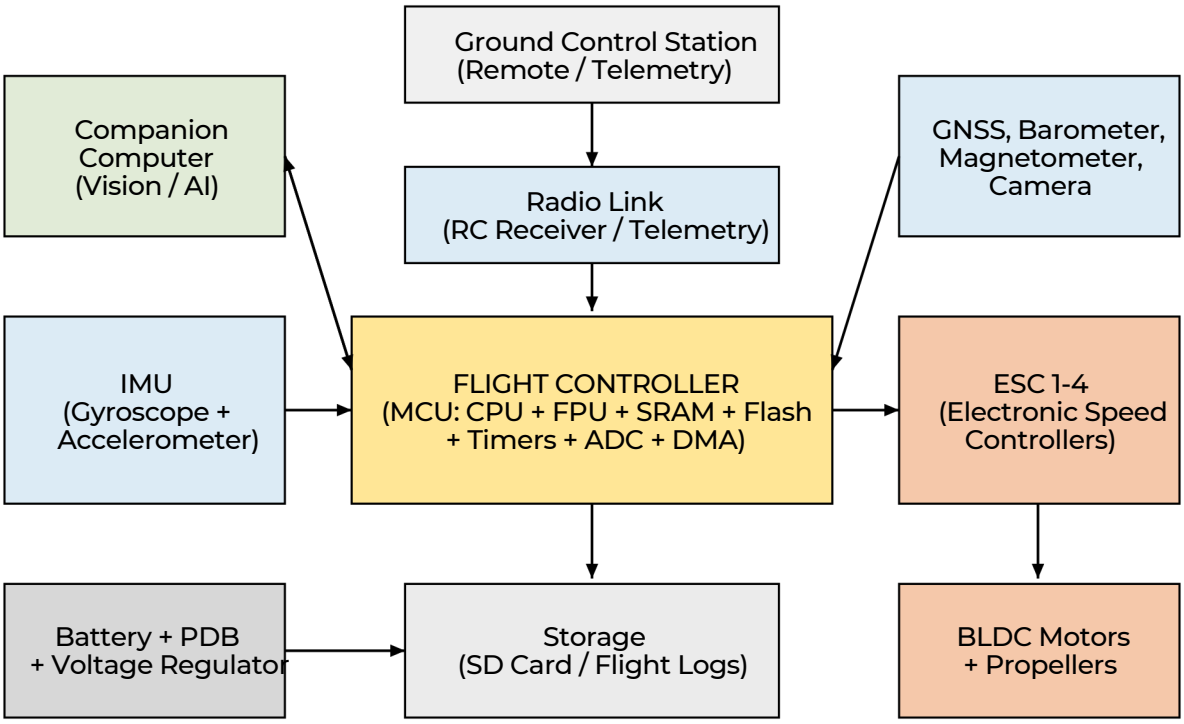


Figure 2: Computer organization of a typical quadcopter drone

This arrangement divides work in a hierarchical manner. The sensor chips perform conversion of physical quantities into digital values; the flight controller performs the time-critical control computation; the ESCs perform fast commutation of the motor phases; and the companion computer performs non-time-critical but heavy computation. Each unit has a clear responsibility, and the interconnections between units are well-defined digital interfaces.

2. Central Processing Unit of the Flight Controller

The CPU of the flight controller is normally a 32-bit ARM Cortex-M core. Like any CPU, it has a **control unit** that fetches and decodes instructions, an **ALU** that carries out arithmetic and logical operations, and a set of **registers** that hold operands, addresses and the program counter. The instruction cycle of fetch, decode, execute and store is repeated

continuously. Pipelining allows several instructions to be at different stages at the same time, which raises the throughput without raising the clock frequency.

A key feature for a drone is the **floating-point unit (FPU)**. Control algorithms and sensor fusion involve many multiplications, divisions, trigonometric functions and square roots on fractional numbers. An FPU executes single-precision floating-point operations in hardware in a few clock cycles, whereas a processor without one must emulate them in software, which is many times slower. Hardware multiply-accumulate and DSP instructions further speed up filters and matrix calculations.

Another important feature is the **interrupt controller**. The CPU does not continuously check each device (polling). Instead, a device signals the CPU through an interrupt request, the CPU saves its present state, executes a short interrupt service routine, and then resumes the interrupted work. Interrupts are used when a new IMU sample is ready, when a byte is received from the GPS, or when a timer period has elapsed.

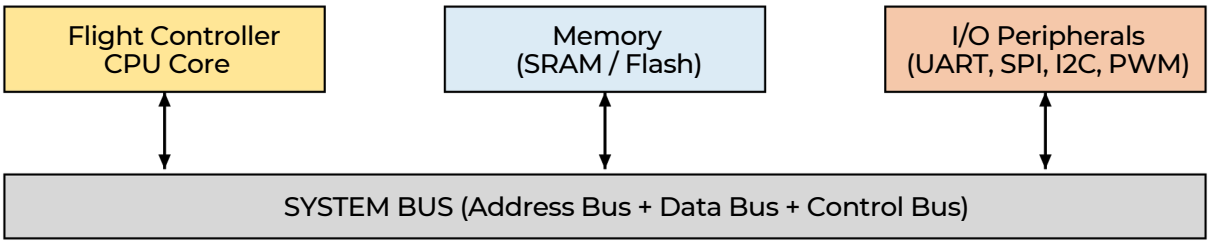


Figure 3: Bus-based organization inside the flight controller

3. Memory Organization

The flight controller uses a hierarchy of memories with different speeds, sizes and purposes.

Memory	Type	Use in the Drone
CPU registers	Flip-flop based, fastest	Hold operands, results, program counter and stack pointer
SRAM	Volatile, fast	Stack, variables, sensor buffers, filter states, real-time data
Flash memory	Non-volatile	Firmware (program code) and constants
EEPROM / Flash parameters	Non-volatile, slow to write	Calibration values, PID gains, configuration settings
SD card / external flash	Non-volatile, large	Flight logs ("black box") and mission data

The program is stored in Flash so that it is retained when the battery is disconnected. At power-up it starts executing from a fixed reset address. Working data live in SRAM, which

is faster than Flash and can be written freely. Memory is also **memory-mapped**: the registers of peripherals such as timers and serial ports appear at fixed addresses in the same address space, so the CPU controls them with ordinary load and store instructions.

The amount of memory is small, therefore the firmware avoids dynamic memory allocation during flight. Fixed-size buffers are allocated at start-up. This avoids fragmentation and keeps the timing of the program predictable, which is a requirement of real-time systems.

4. Input/Output Organization and Buses

A drone has many input/output devices, each connected through a suitable interface. The common interfaces are compared below.

Interface	Type	Typical Devices
SPI	Synchronous serial, high speed, separate chip-select lines	IMU (gyroscope and accelerometer), barometer, SD card
I2C	Two-wire serial bus, addressable, moderate speed	Magnetometer (compass), external sensors, displays
UART	Asynchronous serial	GPS receiver, telemetry radio, companion computer, RC receiver
CAN bus	Differential, robust, multi-node	Industrial drones: GNSS modules, ESCs, power modules
PWM / DShot outputs	Timer-generated digital pulses	Electronic Speed Controllers, servos
ADC inputs	Analog to digital conversion	Battery voltage and current sensing

Different data-transfer techniques are used according to the need. **Programmed I/O** is used for slow devices and configuration. **Interrupt-driven I/O** is used for events such as data-ready signals. **Direct Memory Access (DMA)** is used for fast, continuous transfers, for example reading IMU samples or receiving serial data. With DMA the data move between a peripheral and memory without CPU involvement, so the CPU remains free for the control computation.

5. Digital Principles Applied in the Drone

Every function of the flight controller is based on digital logic. The ALU is built from adders, comparators, shifters and logic gates. Registers and memory cells are made from flip-flops and latches. Multiplexers select among multiple data sources, decoders select a memory location or peripheral from an address, and counters form the heart of timers. **Analog-to-digital conversion.** Battery voltage, current and some sensors give analog outputs. An ADC samples the signal and converts it to a binary number. For an n-bit converter with reference voltage V_{ref}, the smallest voltage step is $V_{ref} / 2^n$. For example, a

12-bit ADC with a 3.3 V reference has a resolution of about 0.8 mV, which is more than enough to measure a battery voltage divided down to the input range.

Timers and PWM. A hardware timer is a binary counter driven by the clock. A comparator compares the counter with a stored value, and the output is set high or low when they match. This produces a pulse-width-modulated (PWM) signal whose width is set by software but generated entirely in hardware, so there is no jitter caused by software execution. Conventional ESC signals use pulse widths of about 1 ms to 2 ms to represent zero to full throttle. Newer digital protocols such as DShot send the throttle value as a digital word, which is immune to analog timing errors.

Number representation. Sensor values are stored as signed binary numbers in two's complement form. The raw output of a 16-bit gyroscope, for example, is a signed integer that is scaled by the sensitivity setting to give an angular rate in degrees per second. The controller converts between integer and floating-point formats when required.

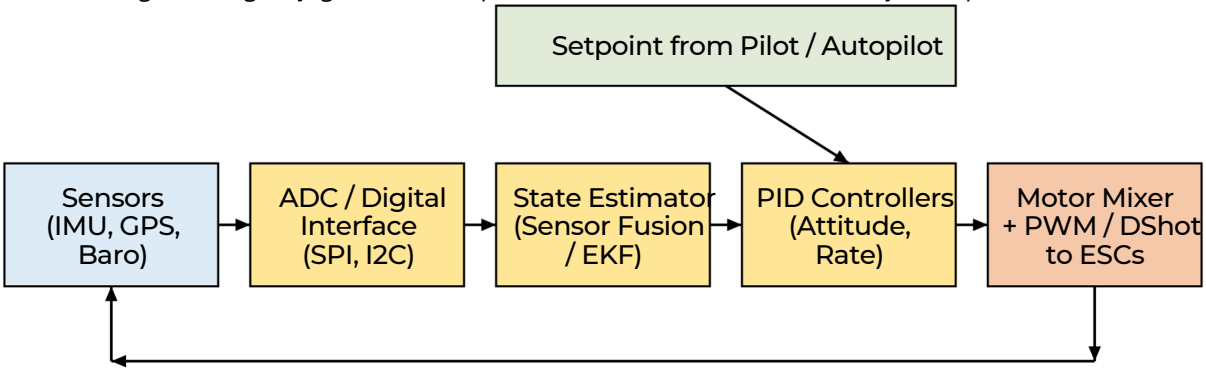
Clock. A crystal oscillator and phase-locked loop supply the system clock. All state changes in the processor take place on clock edges, which is the basis of synchronous sequential design. The clock frequency determines the instruction rate, and a stable clock is essential for accurate timing and communication baud rates.

6. Software and Real-Time Operation

The firmware of a flight controller is normally built on a **real-time operating system (RTOS)** or a carefully scheduled bare-metal loop. Open-source autopilots use RTOS kernels that provide tasks, priorities, timers and message passing. The most urgent task, which is the attitude and rate control loop, has the highest priority. Less urgent tasks such as logging, telemetry and navigation run at lower priority and are pre-empted when necessary.

The central control computation, shown in Figure 4, is a repeated sequence. Sensor data are read, a state estimator combines gyroscope, accelerometer, magnetometer, barometer and GPS data to calculate attitude, velocity and position, and PID controllers calculate the correction required. A **motor mixer** converts the desired thrust and torques into four individual motor commands, which are sent to the ESCs. The inner rate loop typically runs at hundreds to several thousand times per second, while position and navigation loops run more slowly.

Figure 4: Digital flight-control loop (executed hundreds to thousands of times per second)



The state estimation commonly uses a complementary filter or an Extended Kalman Filter (EKF). Both are matrix-and-vector calculations that make heavy use of the multiplier and floating-point unit. The gyroscope gives accurate short-term angular rate but drifts over time, whereas the accelerometer gives a long-term gravity reference but is noisy during manoeuvres; combining the two gives a stable and accurate attitude estimate.

The PID controller computes the control output from the error e between the desired and measured value as $u = K_p e + K_i \int e\,dt + K_d(de/dt)$. In a digital system the integral becomes a running sum and the derivative a difference between consecutive samples. This is simple arithmetic that the ALU and FPU can complete in a few microseconds.

7. Companion Computer and Specialized Processing

When a drone must interpret camera images, build a map or avoid obstacles, the processing requirement is far above what a microcontroller can supply. The drone therefore adds a companion computer, which is normally a multi-core ARM processor with a GPU and sometimes a neural processing unit (NPU). It runs a full operating system such as Linux and communicates with the flight controller through a UART or Ethernet link, using a lightweight message protocol. The companion computer sends high-level commands such as velocity or waypoint setpoints, but the flight controller always keeps the final responsibility for stability.

This arrangement shows the same principle seen in smartphone SoCs: specialized hardware, such as a GPU or NPU, can execute a certain class of workload far more efficiently than a general-purpose CPU. It also isolates the safety-critical control loop from non-deterministic software. If the companion computer crashes, the flight controller can still hold the aircraft stable or return to land.

8. Power Management

The battery, normally a multi-cell lithium-polymer pack, supplies several times the voltage that the digital electronics need. A power distribution board delivers battery power to the ESCs, while voltage regulators, either switching (buck) or linear, produce stable 5 V and

3.3 V rails for the processor and sensors. Power modules measure the battery voltage and current, which are read by the ADC to estimate remaining capacity and trigger a low-battery failsafe. Clean power is essential, because switching noise from motors can disturb sensors and cause incorrect readings, so filtering capacitors and separate regulators are used.

9. Reliability and Fault Tolerance

Because a computer fault in flight can cause damage or injury, drones include several protective mechanisms in their organization:

- **Watchdog timer:** A hardware counter that resets the processor if the software fails to refresh it within a set time, recovering from a software hang.
- **Sensor redundancy:** Larger drones carry multiple IMUs, compasses and GPS units, and the software switches to a healthy one when another fails.
- **Failsafe logic:** Loss of the radio link, low battery or a geofence breach triggers a programmed response such as hovering, landing or returning to the launch point.
- **Error detection:** Checksums and CRCs on telemetry and storage data detect corrupted data, and parity or ECC may protect memory in higher-grade controllers.
- **Pre-arm checks:** The software verifies sensors, calibration, battery and communications before it allows the motors to start.

FINDINGS

The analysis of the computer organization of drones leads to the following findings.

Finding 1: A drone is a complete embedded computer system

A drone contains every unit of a computer: a CPU, memory, input devices, output devices, buses and a clock. The only difference from a desktop computer is that all these units are small, specialized and permanently dedicated to one task, flight control.

Finding 2: Real-time performance matters more than raw speed

The success of the system depends on meeting deadlines in every control cycle. Predictable timing, achieved through hardware timers, interrupts, DMA and prioritised tasks, is more valuable than a high clock rate.

Finding 3: Digital building blocks are directly visible

Counters, comparators, registers, multiplexers, decoders, adders and ADCs appear in the processor, timer and sensor interfaces of the drone. This shows how the theory of digital principles is applied in real devices.

Finding 4: Dedicated hardware reduces the CPU burden

Timers generate PWM signals, DMA moves data, the FPU performs fractional calculations, and the ESC commutates the motors. Each of these offloads work from the CPU and makes the total system more efficient and deterministic.

Finding 5: Sensor fusion turns noisy data into reliable information

No single sensor is good enough alone. Combining gyroscope, accelerometer, magnetometer, barometer and GPS data with filters gives an accurate estimate of the state of the aircraft, and this needs significant arithmetic capability.

Finding 6: Two-level processing improves safety and capability

Separating the flight controller from the companion computer allows intelligent features without risking the stability of flight. Each processor is matched to its own kind of workload.

Finding 7: Fault tolerance is part of the organization

Watchdog timers, redundant sensors, failsafe logic and error detection are built into the system, so that failures lead to a safe outcome rather than a crash.

RECOMMENDATIONS

Based on the analysis and the findings, the following recommendations can be considered for an efficient and reliable drone computer system.

Use processors with hardware floating-point and DSP support

Control and sensor-fusion algorithms are calculation-intensive. A microcontroller with an FPU and multiply-accumulate instructions completes them quickly and with a good margin for deadlines.

Use a real-time operating system with clear task priorities

The fastest control loop should have the highest priority, and logging, telemetry and navigation should run at lower priority so that they never delay flight stabilisation.

Use dedicated hardware timers for motor outputs

Generating PWM or digital motor signals in hardware avoids software jitter and gives precise motor control.

Adopt a two-processor architecture for intelligent drones

A flight controller for stability together with a companion computer for vision and AI gives high capability and keeps the safety-critical loop isolated.

Provide sensor redundancy and failsafe logic

Duplicate IMUs, compasses and GNSS units, together with automatic failsafe actions, improve safety for commercial and long-range missions.

Include watchdog timers and error-detecting codes

Hardware watchdogs and checksums on data detect and recover
communication faults.

Design clean, regulated and filtered power supplies

Separate regulators, filter capacitors and good board layout protect sensors from motor noise and improve the accuracy of measurements.

from software and

Secure communication and firmware

Encrypted links, authenticated commands and secure boot
hijacking, spoofing and unauthorised firmware changes.

can protect drones from

CONCLUSION

The computer organization of drones is an excellent example of how the theory of digital principles and computer organization is applied in a real embedded system. A drone combines a CPU, memory, input and output devices, buses, timers, a clock and a power system in a small and light platform that must operate in real time. Each unit has a clear purpose, and the interconnections between them follow well-defined digital interfaces.

The flight controller, built around a 32-bit microcontroller with a floating-point unit, reads sensors through SPI, I2C and UART, estimates the state of the aircraft using sensor fusion, calculates corrections with PID controllers and sends precise motor commands through hardware timers. Interrupts, DMA and a real-time operating system make sure that the deadlines of the control loop are always met. The companion computer adds vision and artificial intelligence without disturbing the safety-critical loop.

The most important challenge in drone architecture is balancing performance against power, weight, cost, heat and reliability. Features such as dedicated hardware peripherals, efficient memory organization, regulated power supply, watchdog timers and redundant sensors help achieve this balance and make the system safe.

Drone computer organization also shows the strong link between hardware and software. The firmware is written closely around the hardware, and its performance is limited and enabled by the structure of the processor, memory and peripherals. As drones are increasingly used for delivery, agriculture, inspection and autonomous missions, their computer organization will continue to develop, with faster processors, more specialized accelerators and stronger security.

In conclusion, a drone is a flying computer. Studying its organization helps students understand how logic gates, registers, counters, memory, buses and processors combine to form a complete working system that can sense, think and act in real time.

REFERENCES

[1] Mano, M. Morris, and Michael D. Ciletti. *Digital Design: With an Introduction to the Verilog HDL*. Pearson Education.

[2] Mano, M. Morris. *Computer System Architecture*. Pearson Education.

[3] Stallings, William. *Computer Organization and Architecture: Designing for Performance*. Pearson Education.

[4] Hamacher, Carl, Zvonko Vranesic, and Safwat Zaky. *Computer Organization*. McGraw-Hill Education.

[5] Yiu, Joseph. *The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors*. Newnes (Elsevier).

[6] Valavanis, Kimon P., and George J. Vachtsevanos (eds.). *Handbook of Unmanned Aerial Vehicles*. Springer.

[7] Beard, Randal W., and Timothy W. McLain. *Small Unmanned Aircraft: Theory and Practice*. Princeton University Press.

[8] ArduPilot Development Team. ArduPilot Documentation. https://ardupilot.org/dev

[9] PX4 Autopilot Development Team. PX4 User and Developer Guide. https://docs.px4.io

[10] STMicroelectronics. STM32 Microcontroller Reference Manuals and Datasheets. https://www.st.com

[11] Prathyusha Engineering College. Course Material, 241GES301J – Digital Principles and Computer Organization.